# Complemento Práctico: El Arte de Proteger los Datos

Llegamos al concepto que define la robustez de un sistema. El Encapsulamiento no es solo "poner todo en privado", es diseñar quién puede ver y quién puede tocar nuestros datos. En esta guía, analizaremos el peligro de olvidar poner un modificador (la trampa del package-private), aprenderemos a usar los Setters como "patovicas" (guardias de seguridad) que validan datos, y descubriremos cómo hacer que ciertos datos sean de solo lectura.

## Contenido

### La Trampa del Default

¿Qué pasa si no escribo public ni private? Descubriremos el peligroso mundo del package-private y por qué debes evitarlo.

### Smart Setters

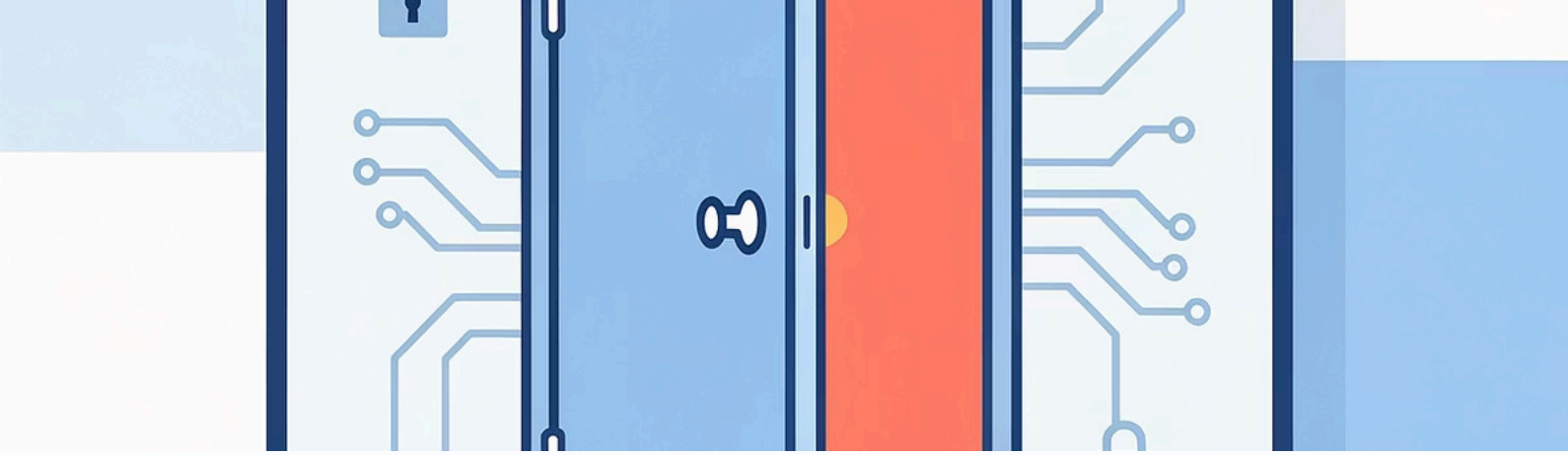
Transformando asignaciones simples en validaciones robustas que protegen la integridad de tus datos.

### Atributos de Solo Lectura

Cómo proteger datos críticos que nunca deberían cambiar después de su creación inicial.

### Productividad en NetBeans

Generación automática de código con Alt + Insert para ahorrar tiempo y evitar errores.



# La Trampa del "Package-Private" (Default)

En el video alertamos sobre un error silencioso. Si declaramos una variable olvidando poner el modificador de acceso, Java no lo hace público ni privado. Le asigna una visibilidad especial llamada **Package-Private** (visibilidad de paquete).

## El Riesgo

Cualquier otra clase que esté en la misma carpeta (paquete) puede acceder y modificar ese saldo directamente. Es como dejar la puerta de tu casa abierta para todos tus vecinos.

```
class CuentaBancaria {
    double saldo;
    // ¡Sin public ni private!
    // Esto es PACKAGE-PRIVATE
}
```

**La Regla de Oro:** Siempre escriban explícitamente `private` a menos que tengan una razón muy específica para no hacerlo. La visibilidad por defecto es una trampa común.

# El Setter como "Guardia de Seguridad"

El apunte muestra cómo crear un setEdad. Pero en la práctica, el Setter es el lugar donde vive la **Lógica de Negocio**. No sirve de nada hacer un atributo privado si luego hacemos un Setter público que deja pasar cualquier cosa. Comparemos dos enfoques diferentes:

## SETTER "BOBO" (Inseguro)

```
public void setSaldo(double saldo) {
    this.saldo = saldo;
}
```

Es casi lo mismo que dejar el atributo público. No hay ninguna protección ni validación. Acepta cualquier valor, incluso negativos o absurdos.

## SETTER "INTELIGENTE" (Encapsulamiento Real)

```
public void setSaldo(double saldo) {
    if (saldo >= 0) {
        this.saldo = saldo;
    } else {
        System.out.println("Error: El saldo no puede ser negativo.");
        // Opcional: Lanzar una excepción (avanzado)
    }
}
```

Este Setter actúa como un verdadero guardián. Valida los datos antes de permitir su ingreso, protegiendo la integridad del objeto.

# Atributos de Solo Lectura

## Read-Only Properties

A veces queremos que un dato se pueda ver desde afuera, pero **nunca modificar**. Ejemplo: El número de cuenta bancaria o el DNI de una persona. Una vez creados, no deberían cambiar.

## La Técnica:

1. Declarar el atributo `private`
2. Crear el método **Getter** (para leer)
3. **NO crear el método Setter**

```
public class Usuario {
    private int dni;
    // Se carga solo en el Constructor

    public Usuario(int dni) {
        this.dni = dni;
    }

    // Solo lectura: Permitimos ver el DNI
    public int getDni() {
        return this.dni;
    }

    // Al no existir setDni(),
    // es IMPOSIBLE cambiarlo desde afuera.
}
```

### Inmutabilidad

Los datos críticos permanecen constantes durante toda la vida del objeto, garantizando consistencia.

### Seguridad

Nadie puede modificar accidentalmente información sensible como números de identificación.

### Confiabilidad

El código que usa estos objetos puede confiar en que ciertos valores nunca cambiarán.

# Productividad en NetBeans

Escribir Getters y Setters a mano es tedioso y propenso a errores de tipeo. Los programadores Java profesionales usamos atajos que nos ahorran tiempo y aumentan la precisión. NetBeans incluye herramientas de generación automática de código que todo desarrollador debe conocer.

### Paso 1

Declaren sus atributos privados:
`private String nombre;`

### Paso 2

Hagan clic en una línea vacía dentro de la clase

### Paso 3

Presionen **Alt + Insert** (o clic derecho → Insert Code)

### Paso 4

Seleccionen "Getter and Setter"

### Paso 5

Marquen todos los atributos y den clic en Generate

**Magia instantánea:** NetBeans escribirá todo el código repetitivo por ustedes en un segundo. Esta técnica es esencial para mantener la productividad en proyectos grandes.

# Comparativa de Niveles de Encapsulamiento

Entender los diferentes niveles de protección es fundamental para diseñar sistemas robustos. Cada nivel tiene su propósito y caso de uso específico.

## Análisis Detallado de los Niveles de Encapsulamiento

### Public (Acceso Total)

- **Descripción:** Cualquier clase desde cualquier paquete puede acceder y modificar directamente.
- **Riesgo:** Máximo. No hay control sobre quién modifica los datos.
- **Uso apropiado:** Constantes públicas (`public static final`) o métodos de utilidad.
- **Ejemplo de problema:** Un saldo bancario público podría ser modificado por cualquier parte del sistema sin validación.

### Package-Private (Default)

- **Descripción:** Accesible solo dentro del mismo paquete, pero sin protección explícita.
- **Riesgo:** Alto. Es fácil olvidar que existe esta visibilidad, lo que puede llevar a accesos no intencionados.
- **Problema principal:** Ocurre por omisión cuando olvidamos escribir el modificador.
- **Por qué evitarlo:** No es una decisión intencional de diseño, sino un error de distracción que reduce el control.

### Protected

- **Descripción:** Accesible en la misma clase, mismo paquete, y clases hijas (herencia).
- **Riesgo:** Medio. Útil para herencia, pero puede exponer demasiados detalles de implementación.
- **Uso apropiado:** Métodos que las subclases necesitan sobrescribir para extender el comportamiento.
- **Consideración:** Es un tema de programación avanzada, usado principalmente en contextos de herencia complejos.

### Private + Getter (Lectura Controlada)

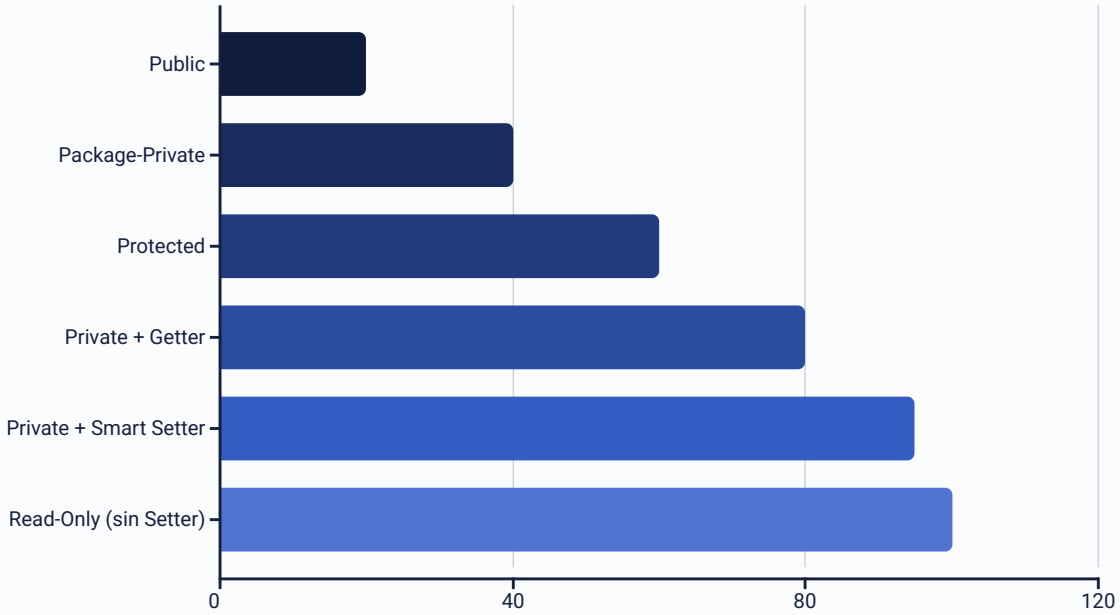
- **Descripción:** Datos privados con un método público para leer su valor.
- **Protección:** Buena. Nadie puede modificar directamente los datos, solo leerlos a través de un método controlado.
- **Ventaja:** Permite un control total sobre cómo se expone la información.
- **Limitación:** Aún permite modificación indirecta si el getter retorna objetos mutables (ej. listas, arrays) en lugar de copias.

## Private + Smart Setter (Escritura Validada)

- **Descripción:** Datos privados con validación lógica implementada en el método de escritura (setter).
- **Protección:** Excelente. Cada intento de cambio pasa por un filtro de validación que asegura la integridad de los datos.
- **Ventaja clave:** Implementa reglas de negocio en un solo lugar, garantizando que el objeto siempre esté en un estado válido.
- **Ejemplo:** Validar rangos, formatos, o lógica compleja antes de aceptar valores, como un saldo bancario que no puede ser negativo.

## Read-Only / Sin Setter (Inmutabilidad)

- **Descripción:** Datos que se establecen una vez, generalmente en el constructor, y nunca cambian después.
- **Protección:** Máxima. Es imposible modificar el valor después de la construcción del objeto.
- **Uso ideal:** Identificadores únicos, fechas de creación, configuraciones fijas, o cualquier dato que no deba alterarse.
- **Beneficio:** Elimina toda una categoría de bugs relacionados con cambios inesperados de estado, simplificando la depuración y el razonamiento sobre el código.




### Regla Práctica

Empiecen siempre con **private**. Solo agreguen Getters y Setters cuando realmente los necesiten. No todos los atributos privados necesitan ser accesibles desde afuera.

# Cierre del Paradigma de Objetos

Hemos completado la transición mental hacia la programación orientada a objetos. Este viaje nos llevó desde conceptos abstractos hasta técnicas prácticas de protección de datos. Ahora poseen las herramientas fundamentales para construir software robusto y mantenible.

### Clases y Objetos

Entendemos la diferencia entre el molde y la galleta. Las clases son plantillas, los objetos son instancias concretas.

### Estado y Comportamiento

Sabemos que los objetos son responsables de sus propios datos. Cada objeto mantiene su propio estado interno.

### Encapsulamiento

Aprendimos a proteger nuestra información usando private y exponiéndola controladamente con Getters y Setters inteligentes.

**¡Están listos para enfrentar desafíos de diseño de software más complejos en la siguiente etapa!**

El encapsulamiento no es solo una técnica, es una mentalidad. Pensar en términos de responsabilidades, protección de datos y contratos claros entre clases es lo que separa el código amateur del código profesional.